

Design Procedure

In this lab, we were tasked with using an audio coder/decoder (CODEC) to generate and filter noises from both an external source and an internal memory. To accomplish this, we were given three tasks. The first involved just accessing and using the CODEC given to us by the lab (playing from external source). The second task was building the ROM that would play a tone from internal memory (MIF). We would use SW[9] to toggle between playing from an external source vs the internal memory. The last task was to make an audio filter that would be toggled via SW[8].

Task #1

In this task, we were instructed to modify the starting kit so that read and write would pass when ready and that writedata (left and right) would update with readdata(left and right). The solution we found was simple. First, we always had writedata update, however it would only pass if read and write were true. We assigned those only if both ready signals are true (Figure 1).

```
assign read = read_ready && write_ready;  
assign write = read_ready && write_ready;  
  
assign writedata_left = readdata_left;  
assign writedata_right = readdata_right;
```

Figure 1: Task 1 Modified assign logic

Task #2

In this task, we were instructed to produce a static tone from memory. After running the python script given to us (this stored samples from sound waveform), we chose G2. After that we generated the appropriate ROM (1 port) that would read the values from the MIF. We ensured that once it reached the max word count, it would loop back to the beginning. To finish off the task, we moved the CODEC from task 1 to a DE1_SoC and set SW[9] to toggle between task 1 and task 2. When SW[9] is on it would play the static tone from memory (2) when it was off it would play from external source (1).

Task #3

For our final task, we were instructed to create an FIR filter. This filter was an averaging filter that removes noise from a sound by averaging the values of adjacent samples. We first

started by building a FIFO buffer specifically or a buffer size of N (this is a parameter that we can change depending on how much filtering is needed. A higher N means more averaging and more filtered of a sound). First, we would take the Log (base 2) of N to make our write pointer (number of bits for writepointer). This ensured it was the appropriate size of the buffer. If update (a signal saying to write into buffer) then we would add the data and increase writepointer (note this is a circular queue so once the write pointer reaches the end, adding one will have it loop back to the start of the queue). At the same time if the buffer is full, dataout gets assigned to where the writepointer is (note this is before the plus one from the update). Our full checks for $N-1$ as the plus one to the writepointer have yet to occur.

```
logic signed [23:0] FIFO_reg [0:N-1];
logic full;
logic [$clog2(N) - 1 : 0] write_pointer;

always_ff @(posedge clk) begin
    if(reset) begin
        for(int i = 0; i < N; i++) begin
            FIFO_reg[i] <= 0;
        end
        full <= 0;
        write_pointer <= 0;
    end
    else if(update) begin
        FIFO_reg[write_pointer] <= data_in;
        write_pointer <= write_pointer + 1;
        if(write_pointer == N-1)
            full <= 1;
    end
end

assign data_out = full ? FIFO_reg[write_pointer] : 0;

endmodule
```

Figure 2: Task 3 FIFO Buffer Logic

Our system is a noise generator and filter. The noises generated are toggled by SW[9]. When SW[9] is active, we play a single tone stored in our system (MIF). When SW[9] is off, we play from an external source. SW[0] is our reset. Finally, SW[8] toggles our filter. When the switch is toggled, it will remove noises from the sound (according to SW[9]) by averaging adjacent samples.

part1.v

This module represents the edited code from task 1. However, we moved the CODEC code given to our DE1_SoC so all this module contains is the logic for passing the read and write data (Figure 1). No Testbench/ ModelSim for this.

Task2ROM.sv

This module instantiates the ROM for task2. This acts as the module that gets the memory for the specific tone we want to play. The ROM data is assign to the writedata (left and right). If the input writeready is true then write becomes true (output telling DE1_SoC that we are writing), and addr is incremented. If addr reaches the size of the MIF file then we have it so it goes back to 0 (the start). If writeready is not true then write goes to 0 (Figure 3).

```
//ROM address and data
reg[16:0] addr;
wire [23:0] rom_data;

//instantiate the ROM module
task2 ROM(.address(addr), .clock(CLOCK_50), .q(rom_data));

//set the left and right data to the ROM values
assign writedata_left = rom_data;
assign writedata_right = rom_data;

always_ff @(posedge CLOCK_50) begin
    if(reset) addr <= 0; //check t

    else if(write_ready) begin
        write <= 1'b1;

        if (addr == 17'd95999) begin
            addr <= 0;
        end

        else
            addr <= addr +1;
    end
    else write <= 1'b0;
end
```

Figure 3: Task2ROM logic

To confirm this behavior, we tested first the reset case. Then we made sure addr was incremented by setting write_ready to, repeating 4092 times or 2^8 of our specified N.(Figure 4). We can also see that rom_data has values from our MIF. Jumping to the end of the simulation, we can see that addr, once it gets to its max, goes back to zero (Figure 5).

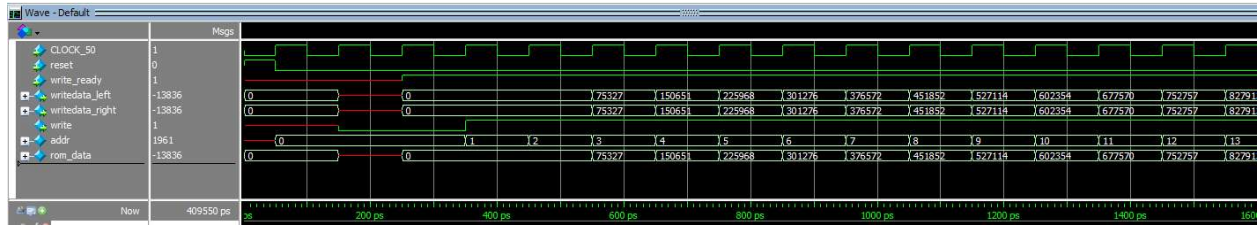


Figure 4: Task2ROM Testbench MODEL SIM (start)

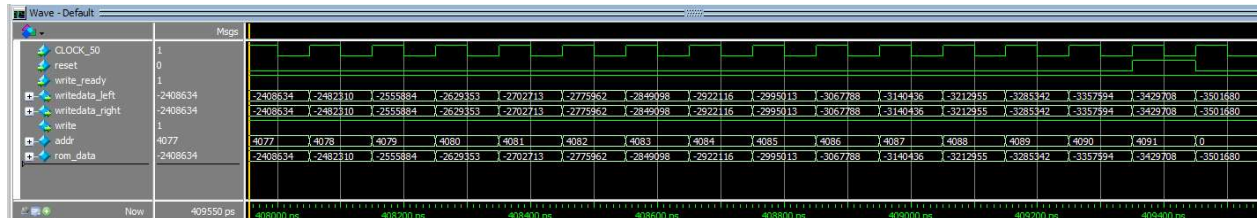


Figure 5: Task2ROM Testbench MODEL SIM (end)

fifo.sv

This module represents the first in first out buffer for FIR in task 3. We built this module specifically for this lab rather than using our code from homework 3. This is because we do not actually use a read_pointer instead the data which is “overflowing” is what is read. At the top of our module is parameter N. This is the rate of filtering (see Task #3 for more information). After creating our input and output logic, we create a FIFO array that will store our values. Follow the logic of if full then data_out passes if not then 0 (Figure3). See Task#3 for more detailed information. We could confirm the logic in our FIR simulation. Looking at Figure 7, we can see that after our reset is applied that fully stays at 0 as the array is being filled with values (Figure 7). Looking at Figure 8, we can see that now the array is filled that the values it passes are the same as the first data_in values from the FIR IE 12, 25, 37... this is also the input_data. Thus, we can confirm that our FIFO buffer works as anticipated (Figure 7 & 8).

FIR.sv

This module is our Finite Impulse Response (FIR) filter. It acts as the module that instantiates the FIFO module. We created our input_data as specified from the lab report (taking the log (base 2) of N to find a bit amount and find average). Data_out is the values passed by the FIFO buffer for us to filter out. Logic is simple: if update then accumulator will get added with the total input and then subtract data_out (Figure 6). Note that data_out will be 0 if the FIFO is not full.


```

localparam int Size = $clog2(N);
logic signed [23:0] input_data;

//divides the data_in by the parameter to take the average
assign input_data = {{Size{data_in[23]}},data_in[23:Size]};

//instantiate the fifo module that stores all the values
fifo FIFO(.clk(clk),.reset(reset), .data_in(input_data),.data_out(data_out),.update(update));

always_ff @(posedge clk) begin
    //reset logic
    if(reset) begin
        accumulator <= 0;
    end

    //when we update (which is synced with the fifo module's update) we want to add the data t
    else if(update)
        accumulator <= accumulator + input_data - data_out;
    end

    assign avg_out = accumulator;
endmodule

```

Figure 6: FIR logic

To confirm that FIR work as intended, we simulated values getting passed through until the FIFO filled. Note: since the FIR relies on the FIFO, we opted to make one testbench that tested their function for clarity. After testing reset we begun passing values of varying size (100, 200, 300, 400, 500, 1000, 4000, 8000...). We can see that during this stage input_data is going to a truncated version of the data_in / size. For 100 we get 12, 200 we get 25, 300 we get 37.... (Figure 7). It is also important to note that update stays at one and accumulator is updated with the previous clock cycles data_in. Now, when the FIFO fills, we can see changes to start subtracting by data_out. Note: data_out is a relatively small number compared to accumulator in the beginning cycles but towards the end of the frame we can see how this begins to make a difference; accumulator peaks at 20250 before decreasing (Figure 8).

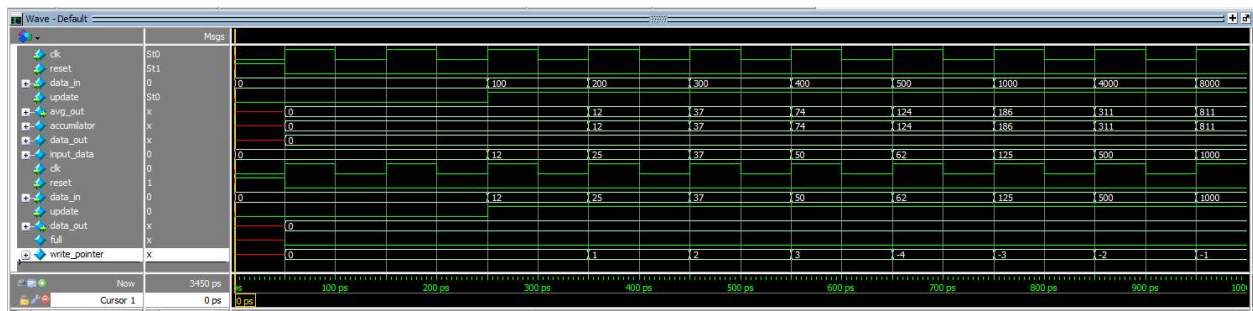


Figure 7: FIR Testbench MODEL SIM (not full)



Figure 8: FIR Testbench MODEL SIM (full)

DE1_SoC

This is our top-level module. It contains the CODEC from task #1 and calls all the other tasks by using the SW[9] (task 1 or 2) and SW[8] (filtered or unfiltered). There are effectively four muxes in our top-level module. The first two choosing between task 1 and task 2 write data according to user input on SW[9] (left and right). This then saves the unmodified writedata (left and right). Then the last two Muxs (much like the first) choose between the unmodified writedata (not filtered) or modified writedata (filtered). We did not include a waveform testing this level as it was mimicking the AUD and I2C signals is unreasonable. The design layout is: Create reset, make wires for task 1 and 2. Instantiate task1 and task 2 writedata. Choose using a mux for the writedata (left and right). Send that to our task 3 FIR. Finally choose through a Mux between filtered and unfiltered writedata. Then send that to the given CODEC from task #1.

```
//reset at switches 0
wire reset = SW[0];
//Wire for task 1 and task 2
logic read_ready, write_ready, read, write;
logic [23:0] readdata_left, readdata_right, writedata_left, writedata_right;
logic [23:0] MOD_writedata_left, MOD_writedata_right, UNMOD_writedata_left, UNMOD_writedata_right;

//Task 1 outputs
logic [23:0] t1_writedata_left, t1_writedata_right;
logic t1_read, t1_write;

//Task 2 outputs
logic [23:0] t2_writedata_left, t2_writedata_right;
logic t2_write;

//Selector Mux based on SW9. If SW9 then do task2 if not then do task 1
assign UNMOD_writedata_left = SW[9] ? t2_writedata_left : t1_writedata_left;
assign UNMOD_writedata_right = SW[9] ? t2_writedata_right : t1_writedata_right;

assign write = SW[9] ? t2_write : t1_write;
assign read = SW[9] ? 1'b0 : t1_read;

//instantiate task1
part1 task1(.CLK(CLOCK_50), .read_ready(read_ready), .write_ready(write_ready), .readdata_left(readdata_left), .readdata_right(readdata_right), .reset(reset), .write_ready(write_ready), .writedata_left(t1_writedata_left), .writedata_right(t1_writedata_right));
//instantiate task2
part2 task2(.CLK(CLOCK_50), .reset(reset), .write_ready(write_ready), .writedata_left(t2_writedata_left), .writedata_right(t2_writedata_right));

//calling FIR from task 3
FIR LEFTtask3(.clk(CLOCK_50), .reset(reset), .data_in(UNMOD_writedata_left), .update(read_ready), .avg_out(MOD_writedata_left));
FIR RIGHTtask3(.clk(CLOCK_50), .reset(reset), .data_in(UNMOD_writedata_right), .update(read_ready), .avg_out(MOD_writedata_right));

//assign final write data left
assign writedata_left = SW[8] ? MOD_writedata_left : UNMOD_writedata_left;
//assign final write data right
assign writedata_right = SW[8] ? MOD_writedata_right : UNMOD_writedata_right;
```

Flow Summary

Flow Summary	
U:/EE371 Homework/Lab 3/lab3starterkit/task2.v	
<<Filter>>	
Flow Status	Successful - Fri May 01 12:39:23 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	part1
Top-level Entity Name	part1
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	1 / 32,070 (< 1 %)
Total registers	0
Total pins	101 / 457 (22 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Experience Report

We found this lab to ...

This lab took us approximately _ hours, broken down as follows:

- Reading – 1 Hour
- Planning – 1 Hour
- Design – 2 Hours
- Coding – 6 Hours
- Testing – 2 Hours
- Debugging – 2 Hours

Total: 14 Hours